



Université numérique
CHEIKH HAMIDOU KANE

Backend

Développement côté serveur et ses fondamentaux

Présenté par : DJIGA SENE

Contenu de la séquence

I. Frameworks backend

II. Mise en place d'un serveur RESTful

III. Utilisation d'un ORM & Spring Data Rest

IV. Outils de test – Postman & cUrl

V. Swagger et Open API



Objectifs spécifiques

A la fin de cette sequence, l'apprenant devrait avoir la capacité de:

- Comprendre les principes et les concepts des frameworks backend.
- Apprendre à mettre en place un serveur RESTful pour créer des APIs web
- Maîtriser l'utilisation d'un ORM pour simplifier la manipulation des données dans une application backend.
- Utiliser des outils de test tels que Postman et cUrl pour tester les endpoints de l'API.
- Explorer l'importance de la documentation des APIs en utilisant des outils comme Swagger ou OpenAPI

Frameworks Backend



Framework Backend

Définition

- Un ensemble d'outils, de bibliothèques et de conventions de codage, fournissant une structure pour développer et gérer des applications web côté serveur de manière plus rapide et normalisée tout en gérant les aspects techniques du développement
- Les principaux avantages de l'utilisation d'un framework logiciel pour le développement sont les suivants:
 - Gain de temps
 - Evolutivité
 - Solidité
 - Sécurité
 - Intégrations

Vue d'ensemble des frameworks (Express, Django, Spring) 1/2

Express

- Connu aussi sous le nom Express.js
- logiciel open-source disponible sous la licence du MIT
- JavaScript comme langage de programmation
- Node.js – applications côté serveur

Django

- Framework open-source basé sur le langage de programmation Python
- Un outil d'administration
- Un système d'authentification, de gestion des mots de passe

Vue d'ensemble des frameworks (Express, Django, Spring) 2/2

Spring Boot

- Plate-forme Java EE
- Framework d'application open-source
- Soutien intégré pour Jetty et Tomcat
- Gestion des dépendances

Etude des frameworks les plus répandus

Tableau comparatif des frameworks

Caractéristiques	Express	Django	Spring
Langage de programmation	JavaScript	Python	Java
Popularité	Très populaire dans l'écosystème Node.js	Très populaire dans la communauté Python	Très populaire dans l'univers Java
Type de framework	Minimaliste et non-opinionné	Fullstack	Fullstack
Architecture	Orienté middleware	MVC	MVC
Flexibilité	Très flexible et extensible	Flexible, avec des conventions strictes	Moins flexible, mais extensible
Performances	Très performant pour les applications légères	Performant pour les applications de taille moyenne à grande	Performant pour les applications d'entreprise

Etude des frameworks les plus Cas d'utilisation répandus

Framework	Type d'application	Cas d'utilisation connus
Express	• Applications Web légères • Applications Single-Page • API REST	• Walmart (Application Mobile) et autres • Uber (gestion des courses et des paiements) • Netflix (parties de l'infrastructure)
Django	• Applications Web Complexes • Applications Backend pour Mobile • Plateformes de Publication	• Instagram (authentification, stockage) • Pinterest (Backend) • Sportify
Spring Boot	• Applications d'Entreprise • Microservices • Applications Web Haute Performance	• Linkedin • Alibaba • Ticketmaster

Mise en place d'un serveur RESTful





Project

☐ Gradle ☐ Groovy ☐ Gradle - Kotlin ☒ **Maven**

Language

☒ **Java** ☐ Kotlin ☐ Groovy

Spring Boot

☐ 3.3.0 (SNAPSHOT) ☐ 3.3.0 (M1) ☐ 3.2.3 (SNAPSHOT) ☒ **3.2.2**
☐ 3.1.9 (SNAPSHOT) ☐ 3.1.8

Project Metadata

Group
Artifact
Name
Description
Package name
Packaging ☒ **Jar** ☐ War
Java ☐ 21 ☒ **17**

Dependencies

ADD DEPENDENCIES... CTRL + B

Spring Web **WEB**

Build web, including RESTful, applications using Spring MVC. Uses Apache Tomcat as the default embedded container.

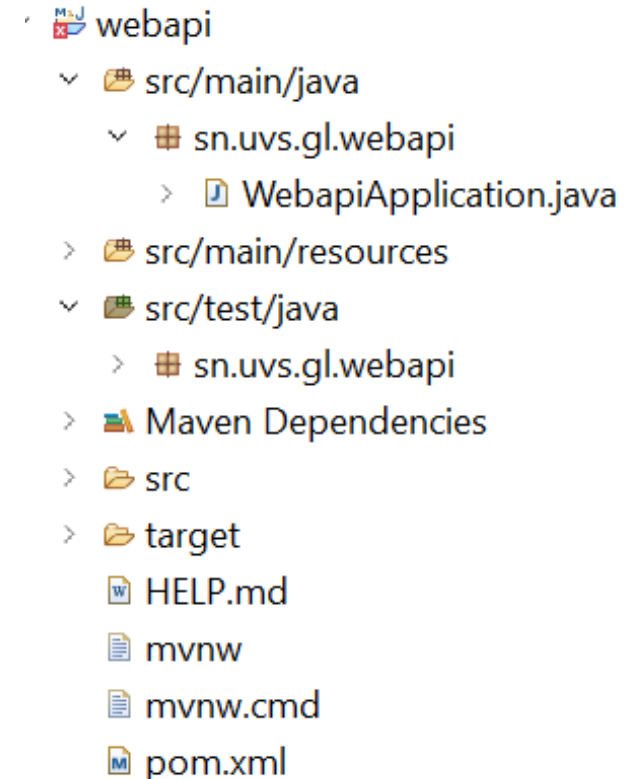
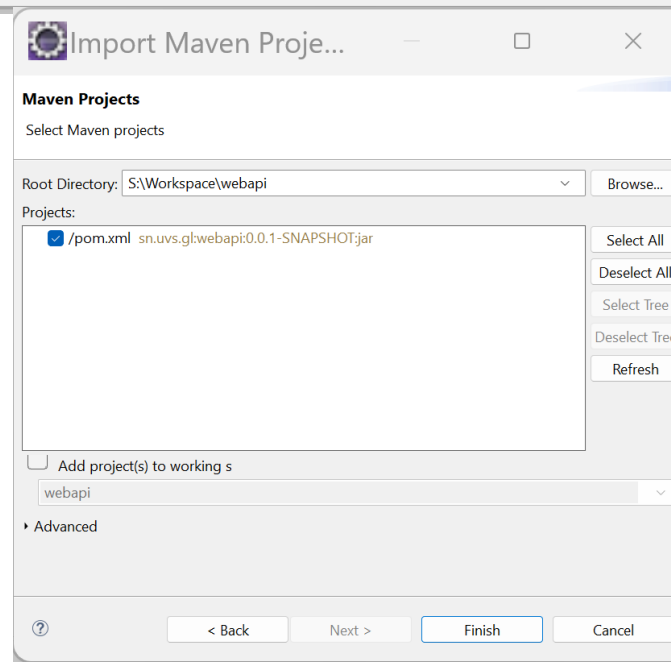
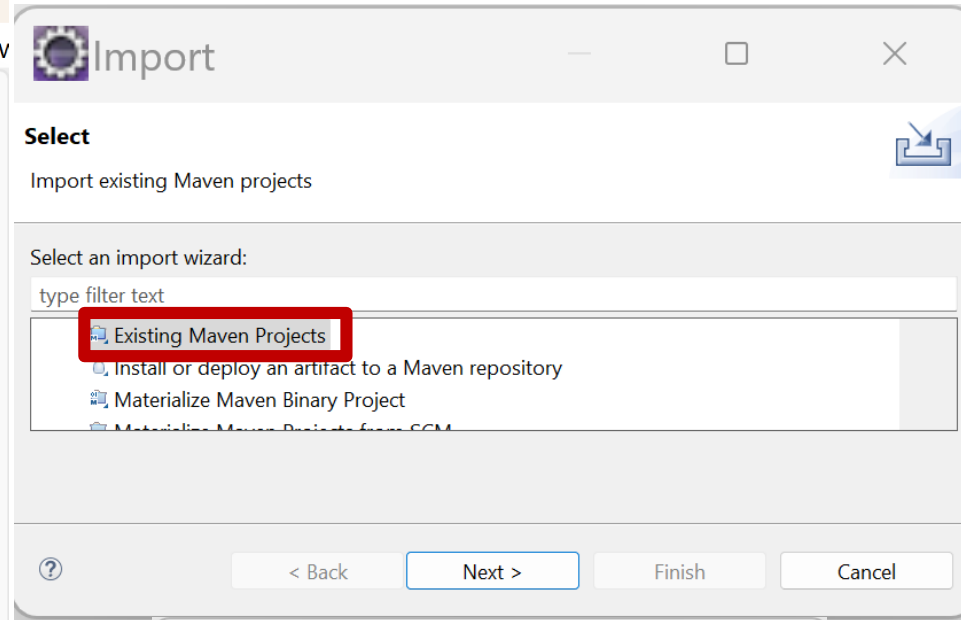
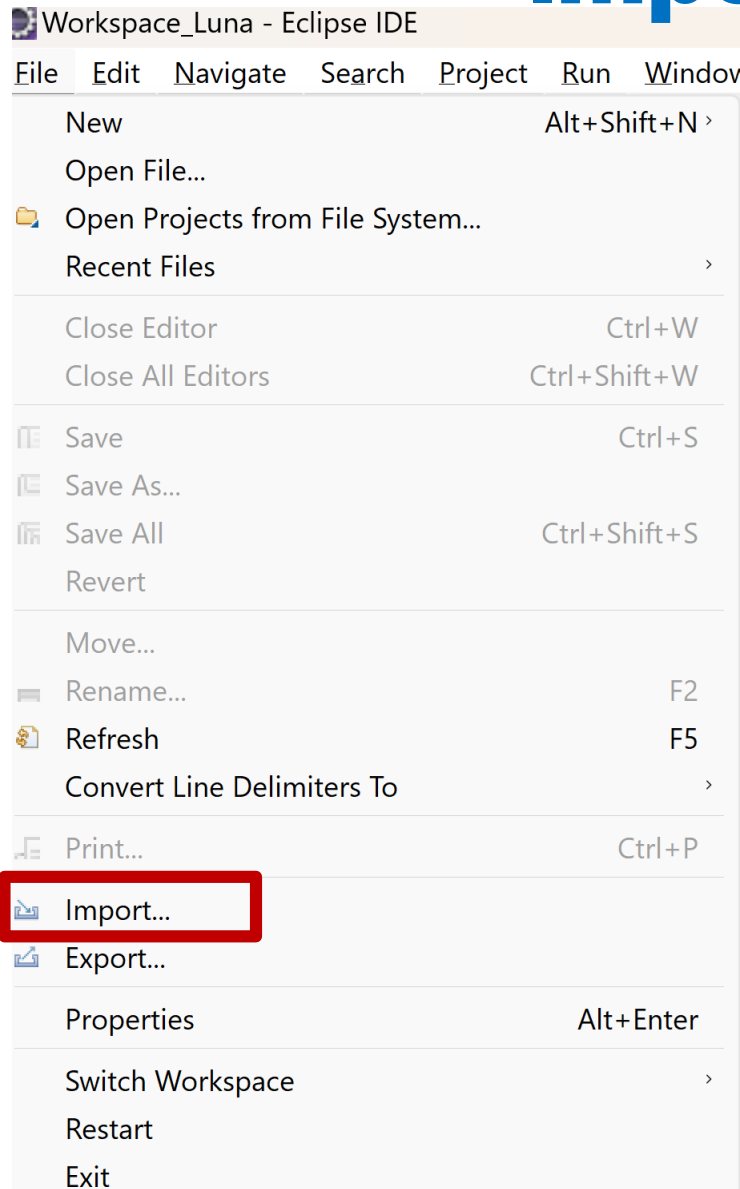


GENERATE CTRL + G

EXPLORE CTRL + SPACE

SHARE...

Importer le projet dans eclipse

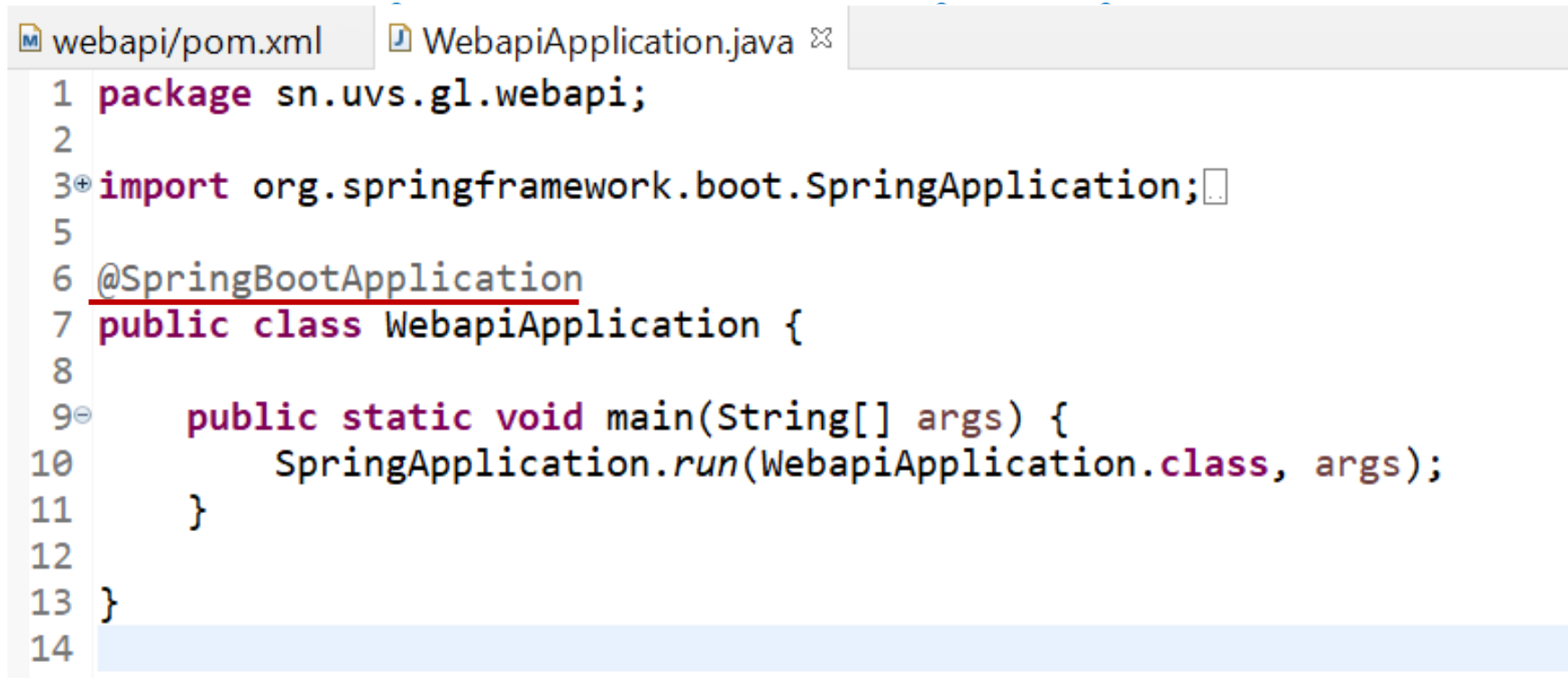


Fichier POM.XML

webapi/pom.xml

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
3     xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 https://maven.apache.org/xsd/maven-4.0.0.xsd">
4     <modelVersion>4.0.0</modelVersion>
5     <parent>
6         <groupId>org.springframework.boot</groupId>
7         <artifactId>spring-boot-starter-parent</artifactId>
8         <version>3.2.2</version>
9         <relativePath/> <!-- lookup parent from repository -->
10    </parent>
11    <groupId>sn.uvs.gl</groupId>
12    <artifactId>webapi</artifactId>
13    <version>0.0.1-SNAPSHOT</version>
14    <name>webapi</name>
15    <description>Demo project for Spring Boot</description>
16    <properties>
17        <java.version>17</java.version>
18    </properties>
19    <dependencies>
20        <dependency>
21            <groupId>org.springframework.boot</groupId>
22            <artifactId>spring-boot-starter-web</artifactId>
23        </dependency>
24
25        <dependency>
26            <groupId>org.springframework.boot</groupId>
27            <artifactId>spring-boot-starter-test</artifactId>
28            <scope>test</scope>
29        </dependency>
30    </dependencies>
31
32    <build>
33        <plugins>
34            <plugin>
35                <groupId>org.springframework.boot</groupId>
36                <artifactId>spring-boot-maven-plugin</artifactId>
37            </plugin>
38        </plugins>
39    </build>
40
41 </project>
42
```

Annotation



```
webapi/pom.xml WebapiApplication.java ✕
1 package sn.uvs.gl.webapi;
2
3+ import org.springframework.boot.SpringApplication;
4
5
6 @SpringBootApplication
7 public class WebapiApplication {
8
9-     public static void main(String[] args) {
10         SpringApplication.run(WebapiApplication.class, args);
11     }
12
13 }
14
```

L'annotation **@SpringBootApplication** équivaut à l'utilisation **@Configuration**, de **@EnableAutoConfiguration** et **@ComponentScan** avec leurs attributs par défaut.

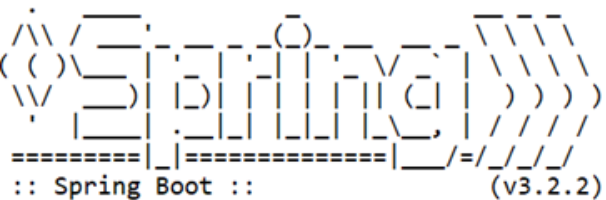
Annotations

- **@Configuration** marque la classe comme une source de définitions de bean pour le contexte de l'application.
- **@EnableAutoConfiguration** indique à Spring Boot de commencer à ajouter des beans en fonction des paramètres de chemin de classe, d'autres beans et de divers paramètres de propriété.
- Normalement, vous ajouteriez **@EnableWebMvc** pour une application Spring MVC, mais Spring Boot l'ajoute automatiquement quand il voit spring-webmvc sur le classpath. Cela marque l'application en tant qu'application Web et active les comportements clés tels que la configuration d'un DispatcherServlet.
- **@ComponentScan** dit à Spring de rechercher d'autres composants, configurations et services dans le paquet hello, lui permettant de trouver les contrôleurs.

Exécution de l'application

**Faire un clic droit sur le projet, puis sur
Run As et cliquer sur Spring Boot App/
Java Application**

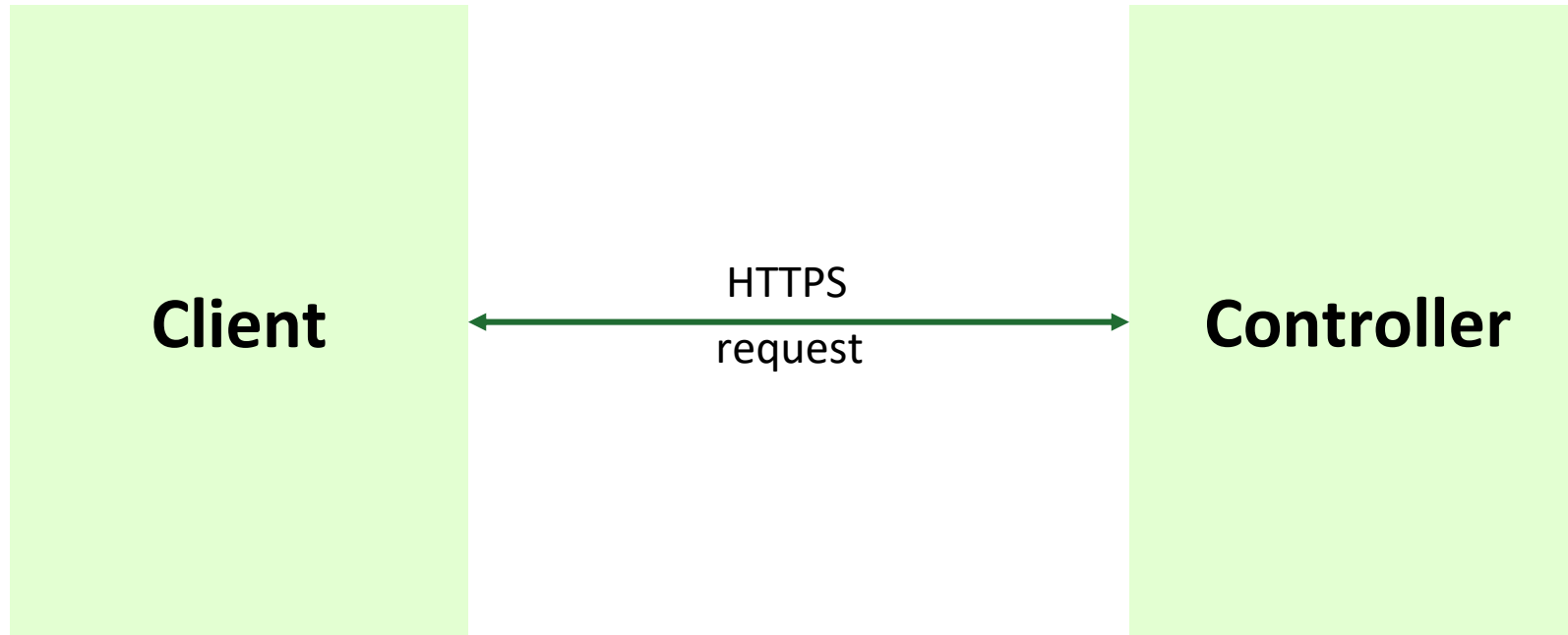
Résultats Attendus :



```
2024-02-21T22:00:17.048-05:00 INFO 25032 --- [main] sn.uvs.gl.webapi.WebapiApplication : Starting WebapiApplication using Java 21.0.2 with PID 25032 (S:\Worksp
2024-02-21T22:00:17.053-05:00 INFO 25032 --- [main] sn.uvs.gl.webapi.WebapiApplication : No active profile set, falling back to 1 default profile: "default"
2024-02-21T22:00:19.189-05:00 INFO 25032 --- [main] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat initialized with port 8080 (http)
2024-02-21T22:00:19.216-05:00 INFO 25032 --- [main] o.apache.catalina.core.StandardService : Starting service [Tomcat]
2024-02-21T22:00:19.216-05:00 INFO 25032 --- [main] o.apache.catalina.core.StandardEngine : Starting Servlet engine: [Apache Tomcat/10.1.18]
2024-02-21T22:00:19.486-05:00 INFO 25032 --- [main] o.a.c.c.C.[Tomcat].[localhost].[/] : Initializing Spring embedded WebApplicationContext
2024-02-21T22:00:19.488-05:00 INFO 25032 --- [main] w.s.c.ServletWebServerApplicationContext : Root WebApplicationContext: initialization completed in 2320 ms
2024-02-21T22:00:20.301-05:00 INFO 25032 --- [main] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat started on port 8080 (http) with context path ''
2024-02-21T22:00:20.328-05:00 INFO 25032 --- [main] sn.uvs.gl.webapi.WebapiApplication : Started WebapiApplication in 3.954 seconds (process running for 7.836)
```

Création d'un service REST 1/3

Flow Architecture

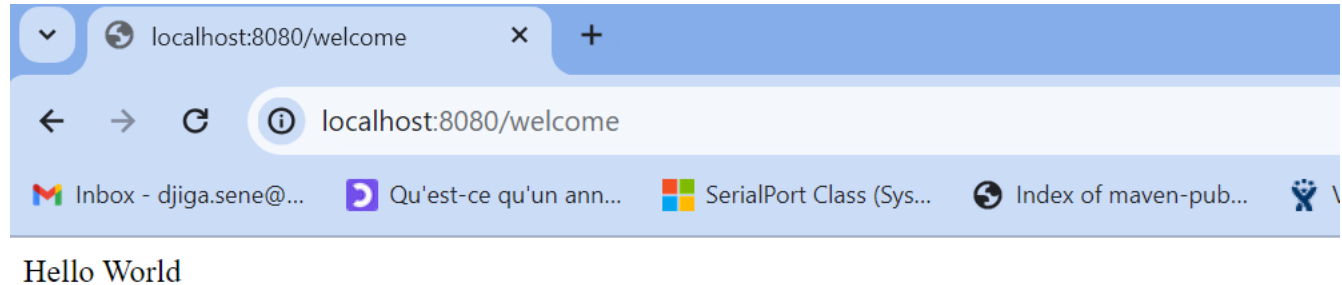


Création d'un service REST 2/3

1. Créer un nouveau package: **sn.uvs.gl.webapi.restcontroller**
1. Créer une classe: **MatiereController**
1. Ajouter l'annotation **@RestController** à la classe
1. Implémenter une fonction: **sayHello** qui retourne la chaîne de caractère **"Hello World"**
1. Rajouter à la fonction précédente l'annotation **@GetMapping("/welcome")**

Création d'un service REST 3/3

```
webapi/pom.xml  WebapiApplication.java  MatiereController.java  ✕
1 package sn.uvs.gl.webapi.restcontroller;
2
3 import org.springframework.web.bind.annotation.GetMapping;
4 import org.springframework.web.bind.annotation.RestController;
5
6 @RestController
7 public class MatiereController {
8
9     @GetMapping("/welcome")
10    public String sayHello() {
11        return "Hello World";
12    }
13 }
14
```



Spring Boot: RestController

@RestController

Il est souvent utilisé pour les APIs qui servent au format JSON, XML, etc.

Spring prend actuellement en charge cinq types d'annotations intégrées pour gérer différents types de méthodes de requêtes HTTP entrantes aux points d'accès de l'API, à savoir **GET**, **POST**, **PUT**, **DELETE** et **PATCH**. Ces annotations sont:

- **GetMapping**
- **PostMapping**
- **PutMapping**
- **DeleteMapping**
- **PatchMapping**

Spring Boot: Annotation

@RequestMapping

L'annotation **@RequestMapping** a les attributs ci-dessous:

- **name**: La Valeur au niveau Type est concaténée avec la Valeur au niveau de la méthode en utilisant '#' comme séparateur.
- **value**
- **path**
- **method**
- **params**
- **headers**
- **consumes**
- **produces**

Spring Boot: Annotation

Spring Boot extraction des paramètres d'une requête:

- **@RequestParam** extrait les valeurs de la chaîne de requête
- **@PathVariable** extrait les valeurs du chemin URI

Spring Boot création de la réponse:

- **@ResponseBody** sérialise la valeur de retour dans le corps de la réponse.
- **@ResponseBody** représente toute la réponse HTTP, y compris le corps, les en-têtes et le statut de la réponse.

Exercice 1

GetMapping

1. Créer un DTO nommé `Matiere(id, code, libelle, credit)`
1. Créer la méthode `getAll()` qui renvoie une liste d'objets **Matiere** dans la classe **MatiereController**
1. Mettre le nom de l'URL "**matieres**" comme argument à **@GetMapping**
1. Lancer l'application et accéder au lien **<http://localhost:8080/matieres>**

localhost:8080/matieres

× +

← → ↺

localhost:8080/matieres

🌐 📄

JSON Données brutes En-têtes

Enregistrer Copier Tout réduire Tout développer 🗑 Filtre le JSON

▼ 0:

id:1

code:"CHM789"

libelle:"Chimie"

credit:5

▼ 1:

id:3

code:"PHY456"

libelle:"Physique"

credit:4

▼ 2:

id:5

code:"MAT123"

libelle:"Maths"

credit:6

Exercice 2

GetMapping avec paramètres

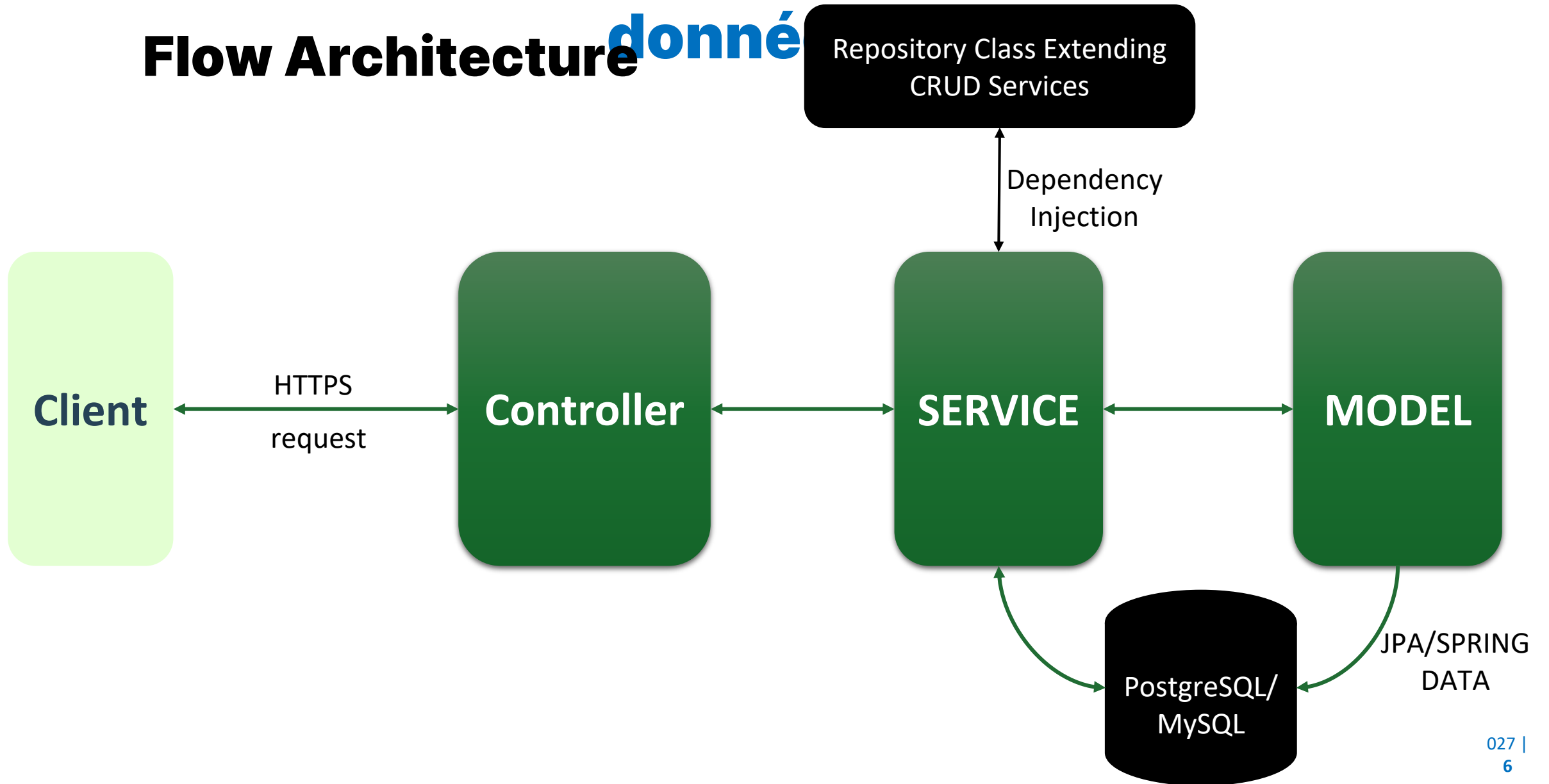
1. Créez la méthode **findByCode()** qui prend en paramètre le code de la matière et renvoie la matière correspondante
1. Ajoutez l'annotation **@GetMapping** avec comme paramètre "**matiere/{code}**" pour définir l'endpoint.
1. Ajouter l'annotation **@PathVariable** avec comme paramètre "**code**" pour récupérer le code de la matière de puis l'URL.
1. Redémarrer l'application et accéder au lien **http://localhost:8080/matiere/<<code>>**
1. Si la matière n'existe pas, renvoyez une **exception not found** avec **ResponseEntity**

SPRING DATA / JPA



Interaction avec une base

Flow Architecture **donné**

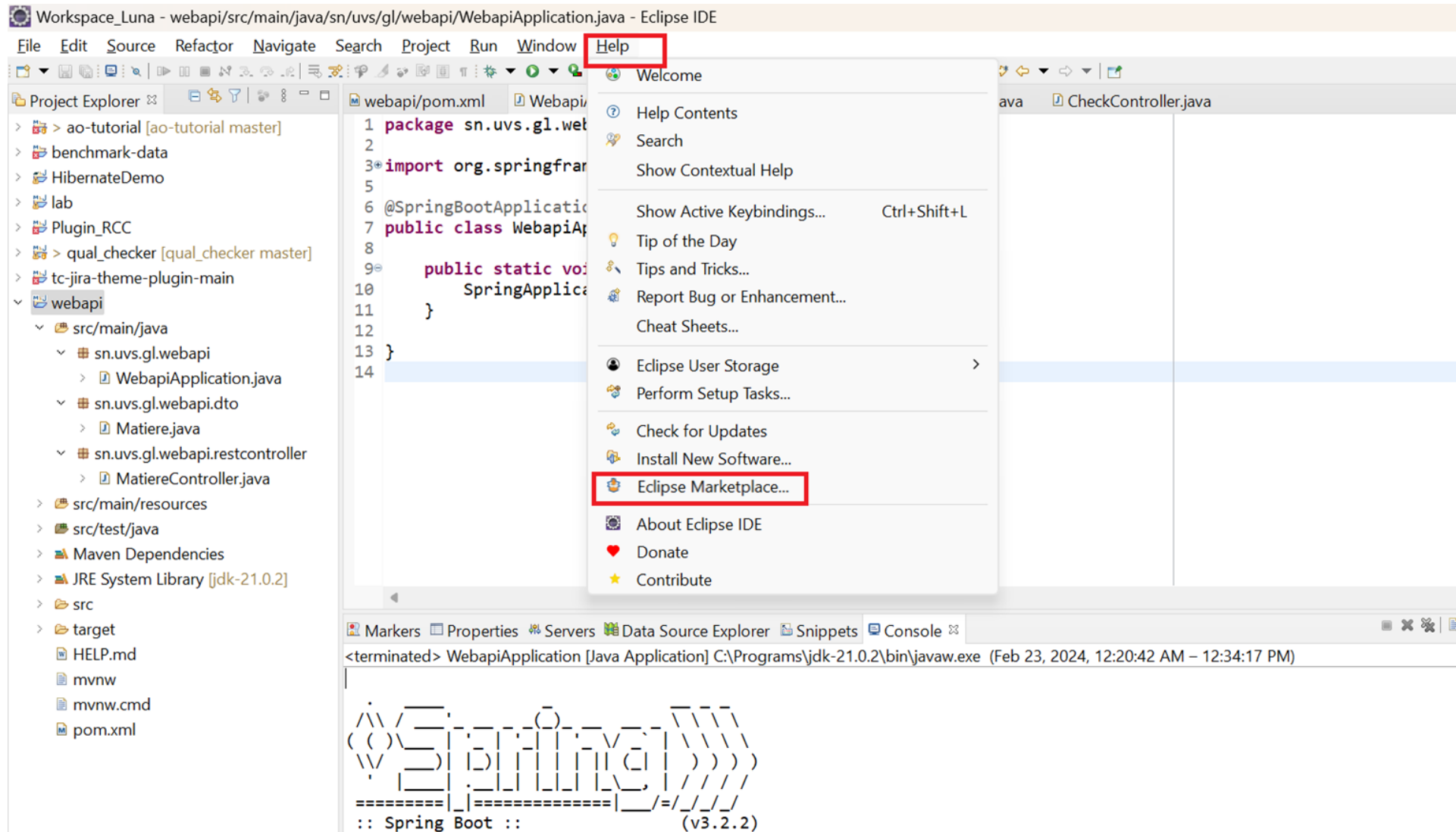


Service et Repository

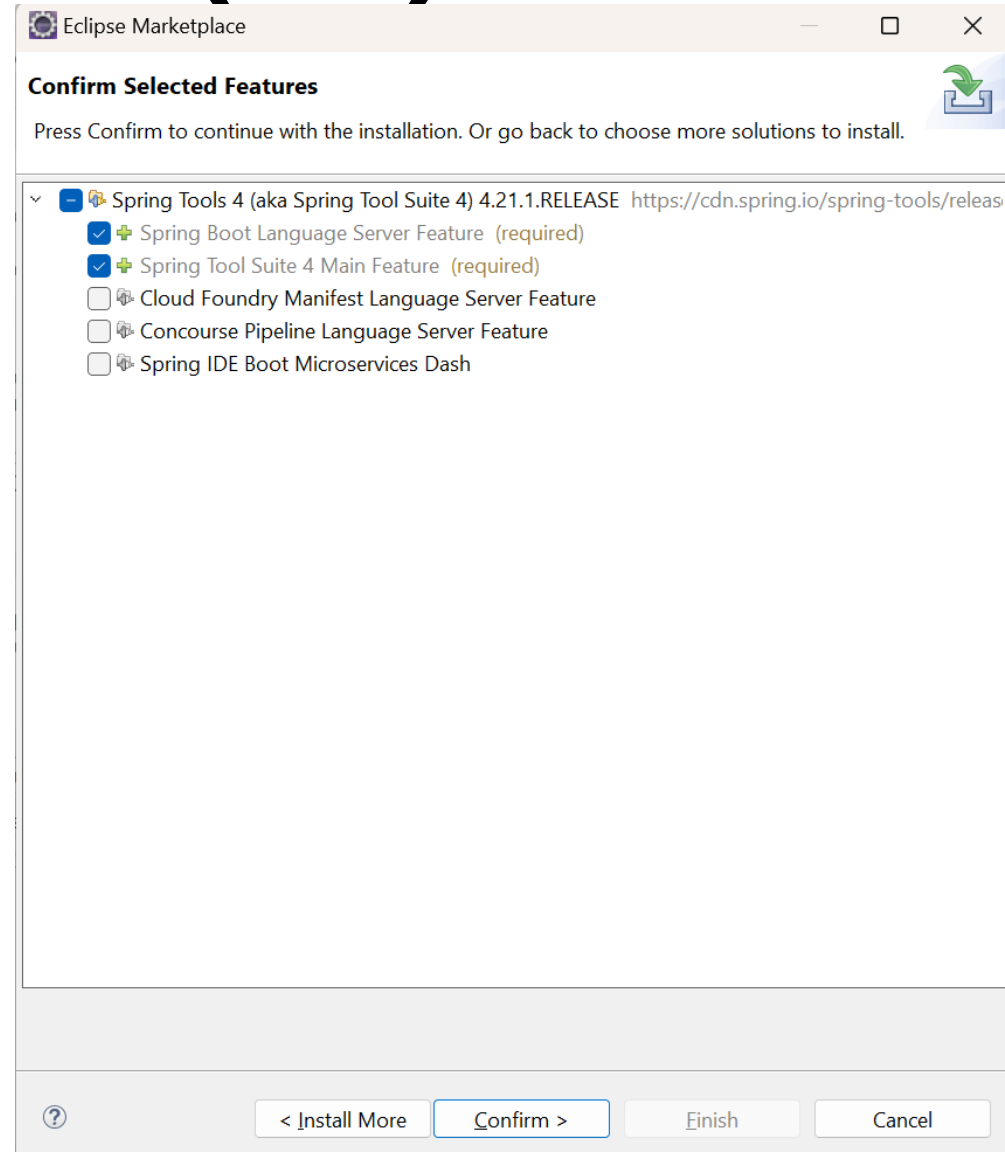
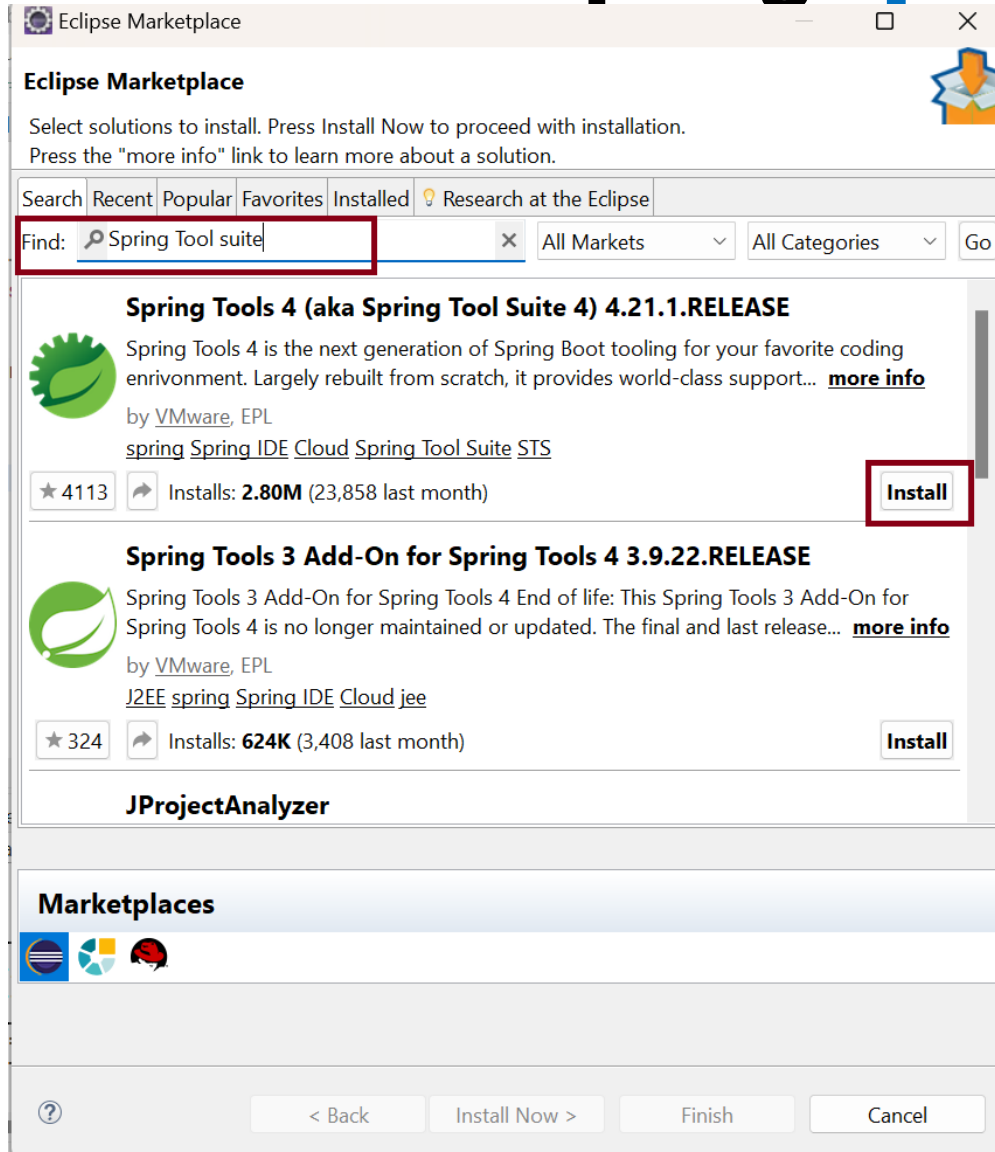
- Une classe service est une classe qui permet de faire des opérations variées comme:
 - Calculer quelque chose
 - Utiliser des repository
 - Le contrôleur décortique le besoin utilisateur et passe la main à la classe service
 - On peut réutiliser la classe Service autant qu'on veut
- Une classe repository permet de faire des traitements relatifs à une base de données

Spring Boot - Ajout de dépendances

1 - Installer Spring Tool Suite (STS)



Spring Boot - Ajout de Installer Spring Tool Suite (STS)



Spring Boot - Ajout de

2 – Récupérer les dépendances depuis le repo

Spring Initializr x Maven Repository: org.springframework x +

mvnrepository.com/artifact/org.springframework.boot/spring-boot-starter-data-jpa/3.2.2

Search for groups, artifacts, categories Search

Indexed Artifacts (37.9M)

Projects (millions)

Year

Popular Categories

- Testing Frameworks & Tools
- Android Packages
- Logging Frameworks
- Java Specifications
- JSON Libraries
- JVM Languages
- Language Runtime
- Core Utilities
- Mocking
- Web Assets
- Annotation Libraries

Home » org.springframework.boot » spring-boot-starter-data-jpa » 3.2.2

Spring Boot Starter Data JPA » 3.2.2
Starter for using Spring Data JPA with Hibernate

License	Apache 2.0
Tags	persistence data spring jpa framework starter
Organization	VMware, Inc.
HomePage	https://spring.io/projects/spring-boot
Date	Jan 19, 2024
Files	pom (2 KB) jar (4 KB) View All
Repositories	Central
Ranking	#206 in MvnRepository (See Top Artifacts)
Used By	2,382 artifacts

Maven Gradle Gradle (Short) Gradle (Kotlin) SBT Ivy Grape Leiningen Buildr

```
<!-- https://mvnrepository.com/artifact/org.springframework.boot/spring-boot-starter-data-jpa -->
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-jpa</artifactId>
  <version>3.2.2</version>
```

Le lien: <https://mvnrepository.com/>

Rechercher en fonction de l'artifact, du groupId, ...

Choisir le type en fonction de la configuration de votre projet

Spring Boot - Ajout de dépendances

Mettre ces deux dépendances dans votre POM
Spring Data JPA

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-data-jpa</artifactId>
</dependency>
```

PostgreSQL

ou

MySQL

```
<dependency>
  <groupId>org.postgresql</groupId>
  <artifactId>postgresql</artifactId>
  <scope>runtime</scope>
</dependency>
```

```
<dependency>
  <groupId>mysql</groupId>
  <artifactId>mysql-connector-java</artifactId>
  <scope>runtime</scope>
</dependency>
```

Interaction avec la base de données

Définir ces informations de connexions à la BD dans application.properties

```
1### DATABASE
2spring.datasource.username=postgres
3spring.datasource.password=123
4
5spring.jpa.properties.hibernate.default_schema=public
6spring.jpa.hibernate.ddl-auto=update
7
8### - POSTGRESQL
9spring.datasource.url=jdbc:postgresql://localhost:5432/matieres
10spring.datasource.driver-class-name=org.postgresql.Driver
11spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.PostgreSQLDialect
12
13### - MySQL
14#spring.datasource.url=jdbc:mysql://localhost:3306/matieres
15#spring.ds.api.driver-class-name=com.mysql.cj.jdbc.Driver
16#spring.jpa.properties.hibernate.dialect = org.hibernate.dialect.MySQL5Dialect
```

Annotation 1/3

@Entity

- Utilisée pour marquer une classe comme entité persistante, permettant à Spring Boot de la gérer via un ORM tel que Hibernate.

@Repository

- Définie une classe comme un composant de persistance, gérant l'accès aux données de la base de données.

@Service

- Pour déclarer une classe comme un composant de service contenant la logique métier de l'appliquatif.

Annotations 2/3

@Autowired

- Injecte automatiquement les dépendances dans les composants de Spring Boot, facilitant l'injection de dépendances sans configuration manuelle.

@Id

- Indique une propriété comme identifiant primaire d'une entité dans une application Spring Boot utilisant JPA. Ceci correspond à la clé primaire de la table .

@Transient

- Permet de déclarer une propriété comme non persistante dans une classe annotée @Entity, indiquant ainsi à JPA de ne pas mapper cet attribut à une colonne de base de données.

Annotations 3/3

@CrudRepository

- Interface de base qui fournit des méthodes CRUD pour accéder et manipuler les données dans une base de données, sans nécessiter d'écriture de code spécifique pour chaque operation CRUD.

@PagingAndSortingRepository

- Interface étendue de **@CrudRepository** qui ajoute des méthodes de pagination et de tri pour faciliter la manipulation de grandes quantités de données dans une application.

@JpaRepository

- Interface étendue de **@PagingAndSortingRepository** spécifique à JPA, qui fournit des méthodes supplémentaires pour travailler avec les entités JPA, telles que la gestion des relations entre entités et les requêtes personnalisées.

Exercice 3

Interaction avec la base de données

1. Créer une entité “**MatiereEntity**” en se référant sur le DTO “**Matiere**”
1. Nommer votre table “matiere” avec l’attribute name de @Entity
1. Lancer votre application
1. Vérifier votre nouvelle table “Matiere” dans votre base de données

Exercice 4

1. Créez une repository pour la classe “**MatiereEntity**” en étendant l’interface **CrudRepository**.
1. Déclarez la méthode “**findByCode**” dans la repository “**MatiereRepository**” pour permettre la recherche d’une matière par son code.
1. Mettez en place un service nommé “**MatiereService**” et ajoutez quelques matières à votre base de données.
1. Adaptez votre contrôleur “**MatiereController**” afin d’utiliser les composants récemment définis.
1. Redémarrer l’application et tester les deux endpoints suivants: <http://localhost:8080/matieres> et <http://localhost:8080/matiere/<<code>>>.

Exercice 5

PostMapping

1. Créez une méthode qui prend en paramètre un objet “MatiereEntity” et qui le stocke dans la base de données.
1. Utilisez l’annotation **@PostMapping** avec l’argument “**matiere/add**” pour définir le nom de l’URL.
1. Ajouter l’annotation **@RequestBody** devant le paramètre “**MatiereEntity**” pour récupérer l’objet passé dans la requête.
1. Téléchargez et installez **Postman**

http://localhost:8080/matiere/add

Save



POST

http://localhost:8080/matiere/add

Send

Params Authorization Headers (8) **Body** Pre-request Script Tests Settings

Cookies

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL

JSON

Beautify

```
1 {  
2   ... "id": 7,  
3   ... "code": "ALG65",  
4   ... "libelle": "ALGEBRE",  
5   ... "credit": 4  
6 }
```

Body Cookies Headers (5) Test Results

200 OK 3.26 s 218 B Save Response

Pretty

Raw

Preview

Visualize

JSON



```
1 {  
2   "id": 7,  
3   "code": "ALG65",  
4   "libelle": "ALGEBRE",  
5   "credit": 4  
6 }
```

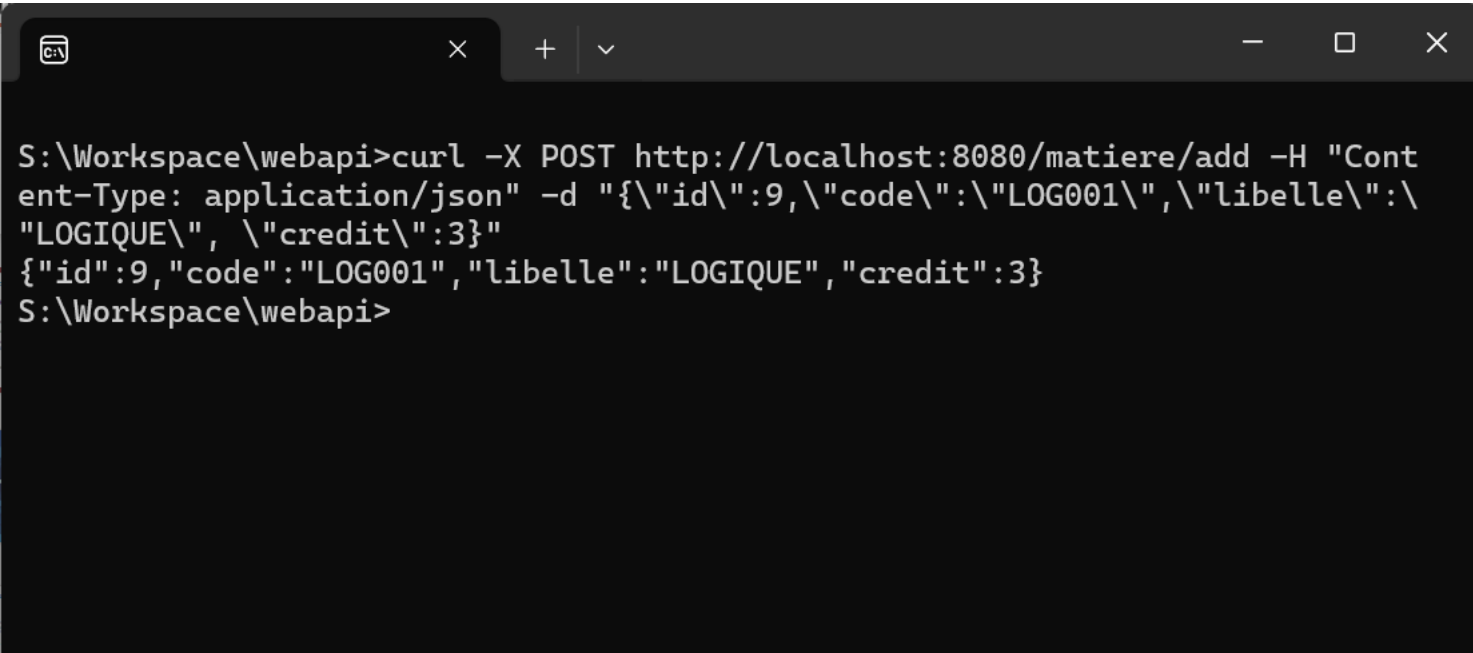
Tester avec cUrl

Curl -X POST

-H "Content-Type: application/json"

-d "{\"id\":9, \"code\":\"LOG001\", \"libelle\":\"LOGIQUE\", \"credit\":3}"

http://localhost:8080/matiere/add



```
S:\Workspace\webapi>curl -X POST http://localhost:8080/matiere/add -H "Content-Type: application/json" -d "{\"id\":9, \"code\":\"LOG001\", \"libelle\":\"LOGIQUE\", \"credit\":3}"
{"id":9,"code":"LOG001","libelle":"LOGIQUE","credit":3}
S:\Workspace\webapi>
```

OpenAPI & SWAGGER



OpenAPI & Swagger

Ajouter la dépendance dans le POM:

```
<dependency>  
  <groupId>org.springdoc</groupId>  
  <artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>  
  <version>2.3.0</version>  
</dependency>
```

Redémarrer l'application

OpenAPI

<http://localhost:8080/v3/api-docs>

The screenshot shows a web browser with four tabs: localhost:8080/matieres, localhost:8080/matiere/PHY456, Swagger UI, and localhost:8080/v3/api-docs. The active tab is localhost:8080/v3/api-docs, which displays the Swagger UI. The browser's address bar shows the URL localhost:8080/v3/api-docs. The Swagger UI interface includes a top bar with tabs for JSON, Données brutes, and En-têtes. Below this is a toolbar with buttons for Enregistrer, Copier, Tout réduire, Tout développer, and a dropdown for Filtrer le JSON. The main content area shows the OpenAPI JSON specification in a tree view on the left and the full JSON on the right. The tree view includes nodes for openapi, info, servers, paths, and components. The JSON on the right is a valid OpenAPI 3.0.1 definition.

```
{
  "openapi": "3.0.1",
  "info": {
    "title": "OpenAPI definition",
    "version": "v0"
  },
  "servers": [
    {
      "url": "http://localhost:8080",
      "description": "Generated server url"
    }
  ],
  "paths": {
    "/matiere/add": {
      "post": {
        "tags": [
          "matiere-controller"
        ],
        "operationId": "saveMatiere",
        "requestBody": {
          "content": {
            "application/json": {
              "schema": {
                "$ref": "#/components/schemas/MatiereEntity"
              },
              "required": true
            }
          }
        },
        "responses": {
          "200": {
            "description": "OK"
          }
        }
      }
    }
  },
  "components": {
    "schemas": {
      "MatiereEntity": {
        "type": "object",
        "properties": {
          "id": {
            "type": "integer",
            "format": "int64"
          },
          "nom": {
            "type": "string"
          },
          "description": {
            "type": "string"
          }
        },
        "required": [
          "id",
          "nom",
          "description"
        ]
      }
    }
  }
}
```

Swagger

<http://localhost:8080/swagger->

The screenshot displays the Swagger UI interface in a web browser. The address bar shows the URL `localhost:8080/swagger-ui/index.html#`. The main heading is "OpenAPI definition" with a "v0" tag and an "OAS 3.0" badge. Below this, a "Servers" section contains a dropdown menu with the selected value "http://localhost:8080 - Generated server url". The API is titled "matiere-controller". A list of endpoints is shown, each with a method (POST or GET) and a path. The endpoints are: `POST /matiere/add`, `GET /welcome`, `GET /matieres`, and `GET /matiere/{code}`. Each endpoint entry has a dropdown arrow on the right side.

OpenAPI definition v0 OAS 3.0

[/v3/api-docs](#)

Servers

http://localhost:8080 - Generated server url

matiere-controller

- POST `/matiere/add`
- GET `/welcome`
- GET `/matieres`
- GET `/matiere/{code}`

SPRING DATA REST



Spring Data REST

- Spring Data REST est une extension de Spring Data qui facilite la creation de services RESTful à partir des interfaces de repository
- Avec Spring Data REST, vous pouvez exposer directement vos repositories en tant que endpoints, pour accéder et manipuler les données via des requêtes HTTP standard.

Rôle:

- Permet de créer rapidement une API RESTful complète
- Réduit le temps de développement
- Favorise une architecture basée sur les microservices

Spring Data REST

- Ajouter cette dépendance dans le POM.

```
<dependency>  
    <groupId>org.springframework.boot</groupId>  
    <artifactId>spring-boot-starter-data-  
rest</artifactId>  
</dependency>
```

Annotation

- Ajouter cette dépendance dans le POM.

```
<dependency>  
    <groupId>org.springframework.boot</groupId>  
    <artifactId>spring-boot-starter-data-  
rest</artifactId>  
</dependency>
```

Servers

http://localhost:8080 - Generated server url ▼

matiere-entity-entity-controller

GET /matiereEntities

POST /matiereEntities

GET /matiereEntities/{id}

PUT /matiereEntities/{id}

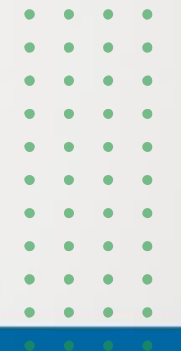
DELETE /matiereEntities/{id}

PATCH /matiereEntities/{id}

matiere-entity-search-controller

Webographie

- Cours 1FullStack2201 Langages et Frameworks Backend 1-Cours Spring Boot du Pr Ousmane SALL
- <https://www.jobtrack.com.au/site/JT/blog/article/rest-define/>
- https://www.researchgate.net/figure/Stateless-and-stateful-data-plane-Adapted-from-132-133_fig1_354302313
- <https://www.xenonstack.com/insights/stateful-and-stateless-applications>
- <https://www.howtographql.com/basics/1-graphql-is-the-better-rest/>
- <https://docs.kaleido.io/kaleido-services/baf/oauth/>



MERCI

